Moon Note - Design System Document

Overview

Moon Note is a fullstack personal knowledge-base application built around a dark, cinematic aesthetic with liquid-glass UI treatments and four procedural GPU-rendered backgrounds. The app functions as a Zettelkasten-style note-taking tool with wiki-style `[[links]]`, Markdown rendering, a D3-powered knowledge graph visualization, and a real-time moon phase widget. Every visual element is purpose-built to create an immersive, contemplative writing environment.

The design philosophy centers on **dark elegance** – a predominantly black canvas with warm amber accents, glass-like translucent panels that float over living procedural backgrounds, and restrained typography that lets the content breathe. The entire interface is a study in restraint: minimal chrome, subtle hover states, and carefully crafted micro-interactions.

Design Tokens

Colors

Token	Hex	Usage
Base/Background	`#000000`	Page background, empty states
Foreground Primary	`#e0e0e0`	Primary text, headings, active states
Dim	`#999999`	Secondary text, hover states
Subtle	`#555555`	Muted labels, inactive icons
Muted	`#333333`	Borders, dividers, ghost text
Accent	`#c8956c`	Buttons, links, interactive highlights
Link	`#d4a574`	Wiki links, external references
Success	`#7dac5a`	Positive states (rarely used)
Danger	`#c45a5a`	Delete actions, error states
Warning	`#e0af68`	Caution indicators
Border	`#1a1a1a`	Panel borders, separators

Liquid Glass Surface Colors (not fixed – derived from backdrop):

- Glass background: `rgba(255, 255, 255, 0.01)` with `luminosity` blend mode
- Glass blur: `backdrop-filter: blur(6px) saturate(140%)`
- Glass strong blur: `backdrop-filter: blur(40px) saturate(150%)`
- Edge highlight: `linear-gradient` white at 25% opacity
- Surface sheen: `radial-gradient` white glow at top edge

Typography

Role	Font	Size	Weight	Color
Body	System stack (-apple-system, BlinkMacSystem-Font, 'Segoe UI', Roboto, 'Helvetica Neue', sans-serif)	14px / 0.875rem	400	`#e0e0e0`
Code/Editor	'SF Mono', 'Fira Code', monospace	13px / 0.8125rem	400	`#999`
Page Title	System stack	20px / 1.25rem	600	`#e0e0e0`
Section Heading (H1)	System stack	24px / 1.5em	600	`#e0e0e0`
Subheading (H2)	System stack	19.2px / 1.2em	600	`#e0e0e0`
Subheading (H3)	System stack	16.8px / 1.05em	600	`#e0e0e0`
Sidebar Label	System stack	10px / 0.625rem	400	`#555`
Brand Mark	System stack	12px / 0.75rem	400	`#888`
Button Label	System stack	12px / 0.75rem	400	varies
Moon Widget Date	System stack	10px / 0.625rem	400	`#555`
Moon Widget Phase	System stack	10px / 0.625rem	400	`#666`
Markdown Body	System stack	14px / 1em	400	`#bbb`
Wiki Link	System stack	14px / 1em	400	`#d4a574`
Graph Node Label	System stack	11px	400	`#777`
Empty State	System stack	14px / 0.875rem	400	`#333`

Line heights:

- Body text: 1.7
- Editor textarea: `leading-relaxed` (approx 1.625)
- Markdown headings: tight (1.2)

Spacing

Token	Value	Usage
Header Height	44px / 2.75rem	Top navigation bar
Sidebar Width	240px / 15rem	Left panel
Content Max Width	672px / 42rem	Editor content area
Panel Padding	12-16px	Internal panel spacing
Button Padding	8px 12px	Standard buttons
Input Padding	8px 12px	Form fields
List Item Padding	8px 12px	Sidebar note items
Gap Between Panels	1px	`gap-px` between sidebar and main
Border Radius (Panels)	0	Sharp edges for layout panels
Border Radius (Buttons)	12px / 0.75rem	`rounded-xl` for action buttons

Generated by Kimi.ai

Table 3 – continued

Token	Value	Usage
Border Radius (Inputs)	8px / 0.5rem	`rounded-lg` for text fields
Border Radius (Small)	6px / 0.375rem	`rounded-md` for tabs

Dependencies

Core Framework

- `react` ^19.2.0 – UI framework
- `react-dom` ^19.2.0 – React DOM renderer
- `react-router` ^7.6.1 – Client-side routing
- `typescript` ~5.9.3 – Type safety
- `vite` ^7.2.4 – Build tool
- `@vitejs/plugin-react` ^5.1.1 – React Vite plugin

Styling

- `tailwindcss` ^3.4.19 – Utility-first CSS
- `tailwindcss-animate` ^1.0.7 – Tailwind animations
- `postcss` ^8.5.6 – CSS processing
- `autoprefixer` ^10.4.23 – Vendor prefixing

UI Component Library

- `@radix-ui/react-*` (40+ packages) – Accessible UI primitives
- `class-variance-authority` ^0.7.1 – Component variants
- `clsx` ^2.1.1 – Conditional classes
- `tailwind-merge` ^3.4.0 – Tailwind class deduplication
- `lucide-react` ^0.562.0 – Icon library

Backend & API

- `@trpc/server` ^11.8.1 – Type-safe RPC server
- `@trpc/client` ^11.8.1 – Type-safe RPC client
- `@trpc/react-query` ^11.8.1 – React hooks for tRPC
- `@tanstack/react-query` ^5.90.16 – Server state management
- `hono` ^4.8.3 – Lightweight web framework
- `@hono/node-server` ^1.14.3 – Hono Node.js adapter
- `drizzle-orm` ^0.45.1 – Type-safe ORM
- `drizzle-kit` ^0.31.8 – Database toolkit
- `mysql2` ^3.14.1 – MySQL driver
- `zod` ^4.3.5 – Schema validation
- `superjson` ^2.2.6 – JSON serialization with Date support
- `jose` 6.1.3 – JWT handling
- `cookie` ^1.1.1 – Cookie parsing/serialization
- `nanoid` ^5.1.6 – Nano ID generation

- `dotenv` ^17.2.3 – Environment variable loading

Data Visualization & Graphics

- `d3` ^7.9.0 – Knowledge graph force simulation
- `d3-force` ^3.0.0 – D3 force-directed layout engine
- `@types/d3` ^7.4.3 – D3 TypeScript types

Markdown & Content

- `react-markdown` ^10.1.0 – Markdown rendering
- `remark-gfm` ^4.0.1 – GitHub-flavored Markdown support

Utilities

- `uuid` ^13.0.0 – UUID generation for notes
- `@types/uuid` ^10.0.0 – UUID types

UI Component Primitives (shadcn/ui ecosystem)

- `@hookform/resolvers` ^5.2.2 – React Hook Form resolvers
- `cmdk` ^1.1.1 – Command palette component
- `embla-carousel-react` ^8.6.0 – Touch carousel
- `input-otp` ^1.4.2 – OTP input component
- `next-themes` ^0.4.6 – Theme provider utility
- `react-day-picker` ^9.13.0 – Date picker component
- `react-hook-form` ^7.70.0 – Form state management
- `react-resizable-panels` ^4.2.2 – Resizable panel groups
- `recharts` ^2.15.4 – Charting library
- `sonner` ^2.0.7 – Toast notification system
- `vaul` ^1.1.2 – Drawer component

Dev Server

- `@hono/vite-dev-server` ^0.19.0 – Hono dev server integration
- `esbuild` ^0.27.2 – JS bundler (used in production build script)

Page: Main App (Root `/`)

Layout

The app uses a **single-page layout** with three vertical zones:

1. **Header Bar** (top, 44px height) – `liquid-glass` treatment, contains brand mark, note count, view toggle tabs, background switcher, and user auth controls
2. **Sidebar** (left, 240px fixed width) – `liquid-glass` panel containing moon phase widget, search input, note list, and action buttons
3. **Main Content Area** (remaining space) – Switches between editor view and graph view

All panels float above a **full-viewport procedural background** that occupies z-index 0. The layout uses `h-screen flex flex-col overflow-hidden` to prevent body scrolling.

Header Bar

- **Left section**: Diamond icon (`◇`) + brand mark + note count
- **Right section**: Editor/Graph toggle buttons + background menu button + user name + logout
- View toggle uses segmented control style: active tab has `bg-white/[0.06]` background
- Background button uses sun/moon icon (`◐`) with tooltip

Sidebar Panel

- **Moon Phase Widget**: Canvas-rendered moon at 48x48px with today's date and phase name below
- **Search Input**: Full-width text field with `bg-white/[0.03]` and placeholder text
- **Note List**: Scrollable list of note titles; selected item has `bg-white/[0.06]` highlight
- **Action Buttons**: "+ New Note" (liquid-glass-strong) and "Manage" at the bottom
- **Manage Mode**: Reveals checkboxes for multi-select with delete/cancel buttons

Main Content – Editor View

- **Tab Bar**: Edit/Preview toggle + source link + delete button
- **Content Area**: Max-width 672px centered
  - Edit mode: title input + content textarea
  - Preview mode: rendered Markdown with `md-body` class + outgoing/incoming link sections

Main Content – Graph View

- Full-width D3 force-directed graph
- Nodes colored with warm amber palette, sized by link count
- Edges: `rgba(200,149,108,0.12)` stroke
- Hover: connected edges brighten, unrelated nodes dim to 15% opacity
- Zoom: 0.15x to 5x with D3 zoom behavior
- Stats overlay: liquid-glass badge showing node and edge counts

Background Menu (Dropdown)

- Fixed position dropdown at `right-4 top-10`
- `bg-[#111]` with `border-white/[0.06]`
- Four background options: Moonlit Ripple, Silk Flow Field, Rain on Glass, Solid Color
- Solid color selected reveals 4 color swatches as round buttons

Interactions

- **Note Creation**: Click "+ New Note" creates empty note, selects it, switches to editor
- **Note Selection**: Click sidebar item loads note in editor; auto-switches to editor view
- **Auto-save**: 400ms debounced save on title/content changes
- **Wiki Links**: Click `[[Title]]` navigates to that note or creates it if non-existent
- **Background Switching**: Dropdown toggles between four procedural backgrounds in real-time
- **View Toggle**: Editor/Graph tabs switch main content area instantly
- **Keyboard**: `Ctrl/Cmd + S` triggers manual save and switches to preview mode
- **Auth Flow**: OAuth redirect to Kimi, callback creates/upserts user, returns session
- **First Login**: Auto-seeds 12 starter notes from server-side `STARTER_NOTES` array

Generated by Kimi.ai

Animations

- **Tab transitions**: Instant (no animation – content switches immediately)
- **Button hover**: `transition-colors` 150ms ease – text color shifts from `#555` to `#999`
- **Glass panel hover**: Subtle opacity shifts on interactive elements
- **Backgrounds**: All four backgrounds run continuous GPU animations at 60fps
  - Moonlit Ripple: Wave simulation with moon reflection, reacts to mouse
  - Silk Flow Field: Noise-driven fabric folds with 3 layered silk planes
  - Rain on Glass: Physics-based raindrop simulation with refraction
  - Solid Color: Static color (no animation)
- **Graph simulation**: D3 force simulation with spring physics, nodes drift gently
- **Loading state**: Simple "Loading…" text centered on screen

---

Page: Login Prompt (unauthenticated gate at `/`)

Layout

When `useAuth` resolves to no authenticated user, `NoteApp` immediately returns `<LoginPrompt />` – a full-screen gate with Moonlit Ripple background. The app is **fully auth-gated**: no notes, editor, or sidebar is rendered until login succeeds.

The prompt is centered vertically and horizontally:

1. Title text (site name)
2. Description text (smaller, dimmer)
3. Liquid-glass-strong panel containing:

- Login prompt label (Chinese: "请先登录以开始使用")
- "使用 Kimi 账号登录" button

A separate `/login` route renders a simpler `Login.tsx` page (standard shadcn Card + Button) accessible directly, though users hitting `/` see the cinematic `LoginPrompt` instead.

Elements

- **Title**: `text-2xl font-medium text-[#e0e0e0]`
- **Description**: `text-sm text-[#666]`
- **Login Panel**: `liquid-glass-strong rounded-xl px-8 py-6`
- **Login Button**: `px-6 py-2.5 text-sm rounded-xl bg-accent/20 text-accent hover:bg-accent/30 transition-colors`
- **Auth Flow**: Redirects to Kimi OAuth, callback at `/api/oauth/callback`, upserts user by `unionId`, returns JWT session in HTTP-only cookie

---

Page: Not Found (`*`, fallback route)

Standard 404 page with app shell layout. Minimal content.

Generated by Kimi.ai

Global Interactions

Liquid Glass Effect (Signature UI Treatment)

Two variants of liquid glass are used throughout:

`liquid-glass` (Standard) – Used on header, sidebar, graph stats overlay:

- `background: rgba(255, 255, 255, 0.01)`
- `background-blend-mode: luminosity`
- `backdrop-filter: blur(6px) saturate(140%)`
- `border: none`
- `box-shadow: inset 0 1px 1px rgba(255, 255, 255, 0.1), 0 8px 32px rgba(0, 0, 0, 0.15)`
- Pseudo-element `::before` creates a subtle edge highlight gradient (25% opacity)
- Pseudo-element `::after` adds a surface sheen with two radial gradients

`liquid-glass-strong` (Prominent) – Used on primary action buttons, login panel:

- Same base but `backdrop-filter: blur(40px) saturate(150%)`
- Different shadow: `4px 4px 4px rgba(0, 0, 0, 0.05), inset 0 1px 1px rgba(255, 255, 255, 0.15)`
- Edge highlight at 30% opacity (slightly stronger)
- Surface sheen at 22% brightness (stronger top glow)

Scrollbar Styling

- Width: 4px
- Track: transparent
- Thumb: `rgba(255,255,255,0.08)` with 2px border-radius
- Thumb hover: `rgba(255,255,255,0.15)`

Background System

Four selectable full-viewport procedural backgrounds, all rendered via WebGL/Canvas at z-index 0:

1. **Moonlit Ripple** (`MoonlitRipple.tsx`)
   - Full WebGL fragment shader simulating moonlit water surface
   - 7-layer wave simulation with FBM noise
   - Moon disc with crater detail rendered procedurally
   - Mouse interaction creates additional ripple waves
   - Fully desaturated to grayscale with 70% luminance factor
   - Runs at 60fps via `requestAnimationFrame`
2. **Silk Flow Field** (`FlowField.tsx`)
   - 3-layer silk fabric simulation with domain warping
   - Kajiya-Kay anisotropic specular reflection model
   - 3 independent silk layers composited back-to-front

Generated by Kimi.ai

- Mouse controls light direction
- Film grain post-processing effect
- ACES tone mapping + gamma correction
- Fully desaturated grayscale output

3. **Rain on Glass** (`RainOnGlass.tsx`)
   - Canvas 2D + WebGL hybrid architecture
   - Physics-based raindrop simulation (up to 900 drops)
   - Drop collision, merging, trail spawning
   - Cityscape background with procedural building silhouettes
   - Water refraction through droplet distortion
   - Desaturated with `saturate(0.7) brightness(0.55)` CSS filter
4. **Solid Color** (no component – CSS background only)
   - Static background color selected from palette
   - Four preset colors: `#000000` (Ink Black), `#1a1a2e` (Deep Blue), `#1a1308` (Brown), `#0d1f0d` (Dark Green)

Wiki Link System

- Syntax: `[[Note Title]]` in Markdown content
- Parsing: `extractLinks()` strips code blocks, finds `\[\[([^\]]+)\]\]` pattern
- Rendering: `wikiLinksToMarkdown()` converts to `[Title](wiki:Title)` for react-markdown
- Click handler: navigates to existing note or creates new one
- Backlinks: `getBacklinks()` finds all notes linking to current note's title
- Graph: `buildGraphData()` constructs nodes + edges for D3 force graph

Moon Phase Widget

- Real-time canvas rendering of current lunar phase
- 48x48px at devicePixelRatio for sharpness
- Pixel-level brightness calculation with crater noise hash
- Terminator line with smooth gradient blending
- Glow effect via radial gradient
- Phase labels: 8 stages from New Moon to Waning Crescent

Core UI Components (shadcn/ui)

The project includes 40+ shadcn/ui components pre-installed in `src/components/ui/`:

Layout: accordion, alert-dialog, alert, aspect-ratio, avatar, badge, breadcrumb, button, button-group, card, carousel, collapsible, dialog, drawer, dropdown-menu, form, hover-card, input-group, label, menubar, navigation-menu, pagination, popover, progress, radio-group, resizable, scroll-area, select, separator, sheet, skeleton, slider, sonner, spinner, switch, table, tabs, textarea, toast, toggle, toggle-group, tooltip

Generated by Kimi.ai

Data Display: calendar, chart, command, empty, kbd

Form Controls: checkbox, input, input-otp, switch

Custom Components (app-specific, not shadcn):

Sidebar (`Sidebar.tsx`)

- Props: `notes`, `selectedId`, `search`, `onSearch`, `onSelect`, `onNew`, `onDeleteMany`
- State: `managing` (boolean), `selected` (Set of IDs)
- Renders: MoonPhase, search input, note list, action buttons
- Manage mode toggles between normal and multi-select views

NoteEditor (`NoteEditor.tsx`)

- Props: `note`, `allNotes`, `onUpdate`, `onDelete`, `onNavigate`
- State: `mode` ('edit' | 'preview'), `title`, `content`, `confirmDelete`
- Auto-save: 400ms debounce via `setTimeout`
- Edit mode: raw title input + content textarea
- Preview mode: ReactMarkdown with custom `a` component for wiki links
- Link sections: outgoing links + backlinks rendered below content

GraphView (`GraphView.tsx`)

- Props: `data`, `onNodeClick`, `selectedNodeId`
- D3 force simulation with 6 forces: link, charge, center, collision, x, y
- Node radius sized by `sqrt(linkCount + 1) * 5.5`
- 8-color warm palette for node fills
- Hover interaction dims unrelated nodes and highlights connections
- SVG zoom with D3 zoom behavior (0.15x to 5x)

MoonPhase (`MoonPhase.tsx`)

- Canvas-based real-time moon rendering
- Phase calculation from synodic month (29.53059 days)
- Pixel-level brightness with crater noise and terminator blending
- Glow radial gradient effect

MoonlitRipple (`MoonlitRipple.tsx`)

- Full WebGL shader background
- Mouse-reactive wave simulation
- Procedural moon disc with craters

FlowField (`FlowField.tsx`)

- 3-layer silk fabric simulation
- Kajiya-Kay specular reflection
- Mouse-controlled lighting

Generated by Kimi.ai

RainOnGlass (`RainOnGlass.tsx`)

- Canvas 2D physics + WebGL refraction
- Up to 900 simulated raindrops
- Procedural cityscape background

Backend Architecture

Database Schema (`db/schema.ts`)

`users` table:

- `id`: serial, primary key
- `unionId`: varchar(255), notNull, unique (Kimi OAuth identifier)
- `name`: varchar(255)
- `email`: varchar(320)
- `avatar`: text
- `role`: enum('user', 'admin'), default 'user'
- `createdAt`: timestamp, defaultNow
- `updatedAt`: timestamp, defaultNow, $onUpdate
- `lastSignInAt`: timestamp, defaultNow

`notes` table:

- `id`: serial, primary key
- `userId`: bigint unsigned, notNull (FK to users.id)
- `title`: varchar(500), notNull
- `content`: text, notNull
- `tags`: json (string array)
- `source`: text (optional citation)
- `createdAt`: timestamp, defaultNow
- `updatedAt`: timestamp, defaultNow, $onUpdate

tRPC Routers (`api/router.ts`)

`auth` router (`api/auth-router.ts`):

- `auth.me` – query: Returns current user from session, or null
- `auth.logout` – mutation: Clears JWT session cookie

`notes` router (`api/notes-router.ts`):

- `notes.list` – authedQuery: Returns all notes for current user, auto-seeds starter notes on first call
- `notes.get` – authedQuery(input: {id: z.number()}): Returns single note by ID

- `notes.create` – authedQuery.mutation(input: {title, content, tags?, source?}): Creates new note
- `notes.update` – authedQuery.mutation(input: {id, title?, content?, tags?, source?}): Updates note
- `notes.delete` – authedQuery.mutation(input: {id: z.number()}): Deletes single note
- `notes.deleteMany` – authedQuery.mutation(input: {ids: z.number().array()}): Bulk delete

`ping` router (`api/router.ts`):

- `ping` – publicQuery: Health check returning `{ ok: true, ts: Date.now() }`

Middleware (`api/middleware.ts`)

- `publicQuery` – No auth required
- `authedQuery` – Requires valid JWT session, injects `ctx.user`; used for both queries and mutations
- `adminQuery` – Extends `authedQuery` with `role === "admin"` check; throws FORBIDDEN for non-admins
- Uses `jose` for JWT verification, cookies for session storage

Auth Flow

1. Frontend calls `getOAuthUrl()` constructing Kimi OAuth URL with `client_id`, `redirect_uri`, `scope=profile`
2. User redirected to Kimi auth page
3. Callback hits `/api/oauth/callback` with authorization code
4. Backend exchanges code for access token, fetches user profile
5. User upserted in `users` table keyed by `unionId`
6. JWT session created and stored in HTTP-only cookie
7. Frontend calls `trpc.auth.me` to get user info
8. `trpc.notes.list` auto-seeds `STARTER_NOTES` if user has zero notes

Data Flow

Frontend hooks (`useAuth.ts`, `useNotes.ts`) wrap tRPC queries/mutations:

- `useAuth`: Queries `auth.me`, exposes `user`, `isAuthenticated`, `isLoading`, `logout`
- `useNotes`: Queries `notes.list`, exposes CRUD operations via tRPC mutations
- All mutations call `utils.notes.list.invalidate()` on success to refresh data
- Notes mapped from DB format (Date objects) to frontend format (timestamps as numbers)

Project File Structure

```
/
├── api/                          # Backend API
│   ├── boot.ts                   # Hono server entry
│   ├── context.ts                # tRPC context builder
│   ├── middleware.ts             # Auth middleware + procedure builders
│   ├── router.ts                 # Root tRPC router
│   ├── auth-router.ts            # Auth tRPC router (me, logout)
│   ├── notes-router.ts           # Notes CRUD router + STARTER_NOTES
```

```
│   ├── queries/
│   │   └── connection.ts         # Drizzle DB connection (getDb())
│   ├── kimi/                     # Kimi OAuth SDK
│   │   ├── auth.ts
│   │   ├── platform.ts
│   │   ├── session.ts
│   │   └── types.ts
│   └── lib/                      # Framework internals
│       ├── cookies.ts
│       ├── env.ts
│       ├── http.ts
│       └── vite.ts
├── contracts/                    # Shared types/constants
│   ├── constants.ts
│   ├── errors.ts
│   └── types.ts
├── db/                           # Database schema
│   ├── schema.ts                 # Users + Notes tables
│   ├── relations.ts
│   └── seed.ts                   # Manual seed script
├── src/
│   ├── main.tsx                  # App entry (StrictMode + BrowserRouter + TRPCProvider)
│   ├── App.tsx                   # Root routes + NoteApp layout
│   ├── App.css                   # Default Vite styles (unused)
│   ├── index.css                 # Global styles + liquid glass + markdown
│   ├── types.ts                  # Note, GraphNode, GraphEdge, GraphData, ViewMode
│   ├── config.ts                 # ALL UI strings + starter notes config
│   ├── const.ts                  # LOGIN_PATH constant
│   ├── store.ts                  # localStorage persistence (unused in auth mode)
│   ├── lib/
│   │   └── utils.ts              # cn() helper (clsx + tailwind-merge)
│   ├── utils/
│   │   └── linkParser.ts         # Wiki link extraction, graph builder, backlink finder
│   ├── hooks/
│   │   ├── useAuth.ts            # tRPC auth hook
│   │   ├── useNotes.ts           # tRPC notes CRUD hook
│   │   └── use-mobile.ts         # Mobile breakpoint detection
│   ├── providers/
│   │   └── trpc.tsx              # tRPC + QueryClient provider
│   ├── pages/
│   │   ├── Home.tsx              # Default Vite page (unused)
│   │   ├── Login.tsx             # Login page component
│   │   └── NotFound.tsx          # 404 page component
│   ├── components/
│   │   ├── Sidebar.tsx           # Left panel (moon, search, notes, actions)
│   │   ├── NoteEditor.tsx        # Editor/preview toggle + content
│   │   ├── GraphView.tsx         # D3 force-directed knowledge graph
│   │   ├── MoonPhase.tsx         # Real-time moon phase canvas widget
│   │   ├── MoonlitRipple.tsx     # WebGL moonlit water background
│   │   ├── FlowField.tsx         # WebGL silk fabric background
│   │   ├── RainOnGlass.tsx       # Canvas+WebGL rain on glass background
```

```
│   │   ├── AuthLayout.tsx        # Auth layout wrapper
│   │   └── AuthLayoutSkeleton.tsx # Auth loading skeleton
│   │   └── ui/                   # 40+ shadcn/ui components
│   └── ...
├── index.html                    # HTML entry point
├── vite.config.ts                # Vite config with Hono dev server
├── drizzle.config.ts             # Drizzle ORM configuration
├── tailwind.config.js            # Tailwind theme + shadcn colors
├── tsconfig.json                 # TypeScript configuration
├── tsconfig.server.json          # Server-side TS config
└── .env                          # Environment variables (generated by init)
```

Notes for Rebuilding

Critical Implementation Details

1. **Liquid glass MUST use the exact CSS** from `index.css`. The effect depends on the precise combination of `backdrop-filter`, pseudo-elements with gradient borders, and radial gradient sheen. Do not simplify or replace with standard `backdrop-blur`.
2. **Four procedural backgrounds are load-bearing visuals**. Each is a self-contained GPU-rendered component (WebGL shaders for Moonlit/Silk, Canvas 2D + WebGL for Rain). They must be lazy-loaded (`React.lazy`) to avoid blocking initial render.
3. **Wiki link syntax `[[Title]]`** is core to the knowledge graph. The parser must strip code blocks before matching, handle case-insensitive lookups, and support click-to-navigate or click-to-create behavior.
4. **Moon phase widget** calculates from synodic month (29.53059 days) with a reference new moon date. It renders pixel-by-pixel on canvas with proper terminator line blending.
5. **D3 graph** uses force simulation with constrained bounds. Nodes are sized by link count. Hover interaction dims unrelated elements via D3 selections.
6. **Starter notes must be kept in sync** between `src/config.ts` (`starterNotes`, currently unused in auth-gated mode) and `api/notes-router.ts` (`STARTER_NOTES`, server-side array auto-seeded into MySQL on first login). The two arrays must contain identical note content.
7. **Auth is strictly required**. The app is fully auth-gated: `useAuth` returning no user causes `NoteApp` to immediately render `<LoginPrompt />` before any notes hooks are called. Unauthenticated visitors cannot view notes, the editor, or the graph. The login flow uses Kimi OAuth exclusively.
8. **Auto-save debounce** is 400ms. The editor uses local state as source of truth during editing, with parent sync only for title renames.
9. **All UI strings are externalized** in `src/config.ts`. Every label, placeholder, and button text comes from config objects (`siteConfig`, `headerConfig`, `sidebarConfig`, `editorConfig`, `graphConfig`, `moonConfig`, `appConfig`).

Tailwind Configuration

The Tailwind config extends the shadcn/ui theme system with CSS custom properties. Key additions:

- `darkMode: ["class"]` (class-based dark mode)
- Sidebar color tokens
- Custom border radius scale: `xs`, `sm`, `md`, `lg`, `xl` derived from `--radius`
- Accordion and caret-blink keyframe animations

CSS Custom Properties (shadcn/ui)

The standard shadcn CSS variables are defined in `:root` but overridden by the dark theme. The actual visible colors come from the explicit hex values in `index.css` and Tailwind arbitrary values like `text-[#e0e0e0]`, not from the shadcn variable system.

Full Prompt for Rebuilding

Complete Reproduction Prompt

> **Build a fullstack personal knowledge-base web application** with the following specifications. Use React 19 + TypeScript + Vite + Tailwind CSS v3 + shadcn/ui for the frontend, and tRPC 11 + Hono + Drizzle ORM + MySQL for the backend. Include Kimi OAuth 2.0 authentication with JWT sessions.
>
> **Visual Design:** Dark cinematic aesthetic with pure black (`#000000`) as the base. All text uses a warm amber accent (`#c8956c`) for interactive elements and `#e0e0e0` for primary content. Secondary text at `#999999`, muted at `#555555`. The signature UI element is a **liquid glass** treatment: panels use `backdrop-filter: blur(6px) saturate(140%)` over `rgba(255,255,255,0.01)` background with pseudo-element edge highlights (`::before` with a 180deg white gradient border at 25% opacity, `::after` with two radial gradients creating a surface sheen). A stronger variant uses `blur(40px) saturate(150%)` for prominent buttons. Thin scrollbars at 4px width with `rgba(255,255,255,0.08)` thumb.
>
> **Layout:** Single-page app with three zones: (1) a 44px liquid-glass header bar containing a diamond icon + brand wordmark + note count on the left, and Editor/Graph toggle tabs + background switcher + user info on the right; (2) a 240px liquid-glass sidebar on the left containing a real-time moon phase canvas widget (48x48px with pixel-level terminator rendering), a search input, a scrollable note list where selected items get `bg-white/[0.06]`, and New Note / Manage buttons at the bottom; (3) a main content area that switches between editor and graph views. All panels float above a full-viewport procedural background at z-index 0 with a 1px gap between sidebar and main.
>
> **Background System:** Four selectable procedural backgrounds rendered via WebGL/Canvas at 60fps: (a) **Moonlit Ripple** – fragment shader with 7-layer wave simulation, procedural moon disc with craters, mouse-reactive ripples, fully desaturated to 70% luminance; (b) **Silk Flow Field** – 3-layer fabric simulation with domain warping, Kajiya-Kay anisotropic specular reflection, mouse-controlled lighting, ACES tone mapping, film grain; (c) **Rain on Glass** – Canvas 2D physics simulation of up to 900 raindrops with collision/merging/trail spawning, procedural cityscape background, WebGL refraction through droplets; (d) **Solid Color** – static CSS background with four preset swatches (black `#000000`, deep blue `#1a1a2e`, brown `#1a1308`, dark green `#0d1f0d`). A dropdown menu at `right-4 top-10` with `bg-[#111]` toggles between options.
>
> **Note Editor:** Split-pane editor with Edit/Preview toggle tabs. Edit mode shows a title input and content textarea with monospace font. Preview mode renders Markdown via `react-markdown` with `remark-gfm` for tables/code blocks/task lists. A custom `a` component handles `wiki:` protocol links.

Content auto-saves with 400ms debounce. `Ctrl/Cmd+S` triggers manual save and switches to preview. Below rendered content, show outgoing links (parsed from `[[Title]]` syntax) and incoming backlinks as clickable tags.

Wiki Link System: Support `[[Note Title]]` syntax in Markdown. Parser strips code blocks first, extracts links via `/\[\[([^\]]+)\]\]/g` regex. On click, navigate to existing note or create a new empty note. Build a bidirectional link graph from all notes for the D3 visualization.

Knowledge Graph: D3 force-directed graph using `d3-force` with 6 forces (link distance 160, charge -400, center, collision, x/y positioning at 0.03 strength). Nodes sized by `sqrt(linkCount + 1) * 5.5`, colored from an 8-color warm amber palette (`#c8956c`, `#d4a574`, `#b8845e`, `#a07050`, `#c0a080`, `#d09060`, `#b09070`, `#c8a888`). Edges at `rgba(200,149,108,0.12)`. Hover dims unrelated nodes to 15% opacity and highlights connected edges to 0.35 opacity. SVG zoom 0.15x-5x. Stats overlay in liquid-glass badge.

Moon Phase Widget: Canvas-rendered real-time moon using synodic month calculation (29.53059 days, reference new moon 2024-01-11). Pixel-level brightness with hash-based crater noise, terminator line smooth blending, and radial gradient glow. Shows current date and phase name below.

Database: MySQL with two tables: `users` (id, unionId, name, email, avatar, role enum user/admin, timestamps) and `notes` (id, userId FK, title varchar(500), content text, tags json, source text, timestamps). Use `serial()` for primary keys, `bigint unsigned` for foreign keys.

API: tRPC routers with Zod validation. `auth` router: `me` query, `logout` mutation. `notes` router: `list` (authed, auto-seeds 12 starter notes on first call), `get`, `create`, `update`, `delete`, `deleteMany`. All note mutations invalidate the list query on success.

Auth: Kimi OAuth 2.0 flow. Frontend constructs OAuth URL with `VITE_KIMI_AUTH_URL` and `VITE_APP_ID`. Callback at `/api/oauth/callback` exchanges code for token, upserts user by `unionId`, creates JWT session in HTTP-only cookie. The app is **fully auth-gated**: `useAuth` hook queries `trpc.auth.me`; when no user is returned, `NoteApp` renders a full-screen `LoginPrompt` with Moonlit Ripple background and a centered liquid-glass-strong login panel. No notes, editor, or graph functionality is accessible without authentication.

Config System: All UI strings externalized in a single `config.ts` file: `siteConfig` (title, description, language), `headerConfig` (brandMark, labels), `backgroundConfig` (4 modes with labels and solid colors), `sidebarConfig` (search placeholder, action labels), `editorConfig` (tab labels, placeholders, link labels), `graphConfig` (stats labels), `moonConfig` (8 phase labels), `appConfig` (empty state), `storageConfig` (localStorage key). Plus `starterNotes` array of 12 seed notes with title, content (Markdown with `[[wiki links]]`), tags, and source.

Starter Notes: 12 comprehensive notes about Large Language Models covering: overview, Transformer architecture, attention mechanism, pre-training & fine-tuning, prompt engineering, RAG, model evaluation, AI safety & alignment, generative AI applications, multimodal models, deep learning fundamentals, reasoning & long context. Each note links to 3-6 other notes via `[[Title]]` syntax. Server-side `STARTER_NOTES` in `api/notes-router.ts` must match client-side `starterNotes` in `src/config.ts`.

Tech Stack: React 19 + TypeScript + Vite + Tailwind CSS v3 + shadcn/ui (40+ components) + tRPC 11 + Hono + Drizzle ORM + MySQL + D3.js + react-markdown + remark-gfm + react-router v7 + Kimi OAuth.